

Fantasy Premier League API – Comprehensive Endpoint Guide

Introduction: The Fantasy Premier League (FPL) provides a set of unofficial but publicly accessible API endpoints (there is no official documentation, so the community has reverse-engineered these) ¹. These endpoints return game data in JSON format, covering everything from players and teams to leagues and live scores. This guide compiles all known FPL API endpoints, describes the data they return, and gives examples and use cases for each. All endpoints share the base URL: `https://fantasy.premierleague.com/api/`. The guide is organized by category (general data, fixtures, player data, team/manager data, leagues, etc.) for clarity.

Endpoint Summary

For quick reference, the table below lists the main FPL API endpoints, the data they provide, and whether authentication is required (most are public and don't need login credentials):

Endpoint	Data Returned	Auth Required
<code>bootstrap-static/</code>	All game metadata – info for all gameweeks, teams, players, etc. ² ³	No
<code>fixtures/</code>	All scheduled fixtures (matches) of the season ⁴ (<i>see parameters below</i>)	No
<code>fixtures/?event={gw}</code>	Fixtures for a specific Gameweek (filter by event ID) ⁴	No
<code>fixtures/?future=1</code>	Upcoming fixtures only (future matches) ⁴	No
<code>element-summary/{player_id}/</code>	Detailed data for a player – past gameweek stats, future fixtures, past seasons ⁵	No
<code>event/{event_id}/live/</code>	Live Gameweek data – real-time stats for all players in that GW ⁶	No
<code>event-status/</code>	Status of current event updates (e.g. bonus points added, standings updated) ⁷	No
<code>dream-team/{event_id}/</code>	Dream team of a given Gameweek (top scoring players in each position) ⁸	No
<code>team/set-piece-notes/</code>	Set piece taker notes for each team (who takes penalties, free-kicks, etc.) ⁹	No

Endpoint	Data Returned	Auth Required
<code>regions/</code>	List of regions (used for regional leagues, e.g. country information) ¹⁰	No
<code>stats/best-classic-private-leagues/</code>	Top 10 private classic leagues (by average of top 5 teams) ¹⁰	No
<code>stats/most-valuable-teams/</code>	Top 10 most valuable fantasy teams (by squad value) ¹¹	No
<code>stats/top/{statistic}/</code>	Top players in a statistical category (e.g. most_transferred_in, most_selected) ¹²	No
<code>entry/{manager_id}/</code>	Manager (user) profile – team name, current points, overall rank, etc. ¹³	No
<code>entry/{manager_id}/history/</code>	Manager's season history – GW-by-GW scores, past seasons, chips used ¹⁴	No
<code>entry/{manager_id}/transfers/</code>	Manager's transfer history for the season ¹⁵	No
<code>entry/{manager_id}/event/{event_id}/picks/</code>	Manager's team lineup and points for a specific Gameweek ¹⁶	No
<code>leagues-classic/{league_id}/standings/</code>	Standings for a classic league (total points league) ¹⁷	No
<code>leagues-h2h/{league_id}/standings/</code>	Standings for a head-to-head league ¹⁸	No
<code>leagues-h2h-matches/league/{league_id}/?event={gw}</code>	H2H league match results for a given Gameweek ¹⁹	No
<code>league/{league_id}/cup-status/</code>	Cup tournament status for a league (knockout bracket info) ²⁰	No
<code>my-team/{manager_id}/</code>	(Auth) Current authenticated user's squad, chips, and transfers ²¹	Yes
<code>me/</code>	(Auth) Authenticated user's profile (useful to get your own <code>manager_id</code>) ²²	Yes
<code>entry/{manager_id}/transfers-latest/</code>	(Auth) Latest transfers in the most recent Gameweek for the user ²³	Yes

Below, we dive into each category of endpoint, describing the JSON structure, key fields, example outputs, and typical use cases.

General Game Data Endpoints

1. `bootstrap-static/` – Full Game Data Dump

Description: The `bootstrap-static` endpoint is the primary source of **overall game information**. It returns a large JSON object with multiple sections covering the entire season's configuration and data ². Key sections include:

- **events:** List of all 38 Gameweeks (events) with details like deadlines and average scores. For each GW, it provides info such as the average score, highest score, top scoring player, most transferred in/out, most captained, etc. ². (Use case: to iterate through gameweeks, or to display overall GW stats and schedule.)
- **game_settings:** Global game rules and settings (scoring system, squad size, chip rules, etc.) ²⁴. (This is mostly static – developers might not need this often, since rules seldom change.)
- **phases:** Season phase info (e.g. segments of the season, not commonly used) ²⁴.
- **teams:** Information on all 20 Premier League teams, including team code, short name, strength (difficulty ratings), etc. ²⁵. (Use case: mapping team IDs to names, team strength for fixture difficulty calculations.)
- **total_players:** Total number of players (FPL managers) overall ³.
- **elements:** List of all FPL **players** (“elements”) with their details and current stats ³. This includes each player's ID, name, team, position, price, ownership percentage, and scores (points, goals, assists, etc.), as well as ICT index data. Essentially, this is the full player dataset. (Use case: populating a database of all players and their current stats; powering draft or analytics apps with up-to-date player info.)
- **element_stats:** Definitions of statistical fields for players (e.g. which fields count as goals, assists, bonus etc.) ²⁶.
- **element_types:** Definitions of player positions (e.g. Defender, Midfielder, etc., along with position-specific info like squad limit) ²⁷.

Because `bootstrap-static/` provides **almost everything** needed to get started, a common practice is to call it once at startup of an app or agent to cache the data. For example, a backend service might hit this endpoint to load all players into a database with their initial stats and prices ³. The data is updated periodically by FPL (especially player stats and prices, which update after each match or gameweek).

Example: The snippet below shows a very simplified example of the structure (truncated for brevity):

```
{
  "events": [
    { "id": 1, "name": "Gameweek 1", "average_entry_score": 57, "finished": true, ... },
    { "id": 2, "name": "Gameweek 2", "average_entry_score": 55, ... },
    ...
  ],
  "teams": [
    { "id": 1, "name": "Arsenal", "short_name": "ARS", "strength": 4, ... },
    { "id": 2, "name": "Aston Villa", "short_name": "AVL", ... },
  ]
}
```

```

    ...
  ],
  "elements": [
    { "id": 254, "team": 1, "web_name": "Saka", "element_type": 3, "now_cost":
85, "total_points": 206, ... },
    { "id": 300, "team": 2, "web_name": "Watkins", "element_type": 4,
"now_cost": 78, "total_points": 170, ... },
    ...
  ],
  "total_players": 11234567,
  ...
}

```

Note: The `game_settings` data can also be retrieved from an unofficial endpoint `game-settings/` (which returns the same rules info) ²⁸, but since it's already contained in `bootstrap-static`, most developers just use the latter.

2. `event-status/` – Event Update Status

Description: This endpoint indicates the **status of real-time updates** in the current Gameweek ⁷. FPL updates certain data in phases (for example, live scores and bonus points during matches, and final league standings after all matches). The `event-status/` JSON tells you whether: bonus points have been added, automatic substitutions processed, leagues updated, etc., for the ongoing or most recent Gameweek ⁷.

Key fields often include `status` (with flags like `"bonus_added": true/false`, `"leagues_updated": true/false` for the current event). This is useful for apps or multi-agent systems that need to know if the final scores are ready – for instance, a service might wait for `bonus_added = true` before calculating dream teams or updating leaderboards. (*Use case: Polling this endpoint after matches to trigger final point calculations or notifications once everything is finalized.*)

3. `regions/` – Regions Information

Description: Returns a list of **regions** used by FPL, likely including countries or geographic regions for international leagues ¹⁰. Each region object may contain an `id`, `name` (e.g. "England", "Norway", "Bangladesh", etc.), and possibly an ISO country code or other details. This is used to support the "Nationality/Region leagues" – FPL automatically places users into a league for their country/region based on this data.

While not commonly used in most applications, this could be useful if you want to present or filter users by region or list the official country leagues. (*Use case: If building a global dashboard, you might use this to allow filtering top players by country league, etc.*)

Fixture Endpoints

4. fixtures/ – Season Fixtures

Description: The `fixtures/` endpoint returns an array of all scheduled **matches (fixtures)** in the Premier League season ⁴. Each fixture object includes details like: fixture `id`, the Gameweek (`event`) it belongs to, the home (`team_h`) and away (`team_a`) team IDs, the scheduled kickoff time, and difficulty ratings for each side (`team_h_difficulty` and `team_a_difficulty`) ²⁹. For completed matches, it also includes the final score and some match stats.

Important fields within each fixture object:

- `event` – the GW number (links to the `events` in bootstrap data) ²⁹.
- `team_h` / `team_a` – the IDs of the home and away teams (refer to `teams` data to get names) ²⁹.
- `team_h_score` / `team_a_score` – goals scored by each team (present if match is finished).
- `team_h_difficulty` / `team_a_difficulty` – FPL's fixture difficulty rating for each team in this match ³⁰.
- `finished` (and `started`) – booleans indicating match status.
- `stats` – an array of match stats that impact FPL scoring ³¹. This includes sub-objects for goals, assists, yellow_cards, etc., each containing a list of players (`element` IDs) and the `value` of that stat. For example, under `goals_scored` you might see which players scored and how many ³².

You can **filter** this endpoint by adding query parameters:

- `?event={gw}` – returns only fixtures for a specific Gameweek ⁴. For example, `/fixtures/?event=7` gives all matches in GW7.
- `?future=1` – returns only future (upcoming) fixtures ³³. `future=0` returns all fixtures (past and future), which is the default. Essentially, `future=1` acts as a Boolean flag to get upcoming matches only ³³.

One important aspect: **past vs future fixtures data**. If a fixture is in the future (not yet played), the API will return a simpler object – basically the schedule info (teams, date, etc.) without player stats. If the fixture is completed, the object will include the full `stats` breakdown as noted above ³⁴ ³⁵. In other words, **future fixtures have summary info, past fixtures have detailed stats** ³⁴ ³⁶.

Use cases: The fixtures data is useful for generating match schedules, doing fixture difficulty analysis (by cross-referencing team difficulty ratings), or updating results. For example, a backend service might fetch `/fixtures/` to know all upcoming matches and then schedule score updates. Multi-agent systems can use this to assign tasks (one agent monitors upcoming fixtures, another calculates predictions, etc.).

Example: A fixture JSON example (illustrative format) –

```
{
  "id": 13,
  "event": 2,
  "finished": true,
  "team_h": 3,                // Home team ID (e.g., 3 = Bournemouth)
```

```

"team_h_score": 2,
"team_h_difficulty": 3,
"team_a": 7, // Away team ID (e.g., 7 = Liverpool)
"team_a_score": 1,
"team_a_difficulty": 4,
"stats": [
  {
    "identifier": "goals_scored",
    "h": [ { "element": 101, "value": 1 }, { "element": 102, "value": 1 } ],
    "a": [ { "element": 205, "value": 1 } ]
  },
  {
    "identifier": "assists",
    "h": [ { "element": 103, "value": 1 } ],
    "a": [ { "element": 206, "value": 1 } ]
  },
  ...
],
"kickoff_time": "2025-08-14T19:00:00Z",
...
}

```

This shows a finished match where home team (ID 3) scored 2 goals (players 101 and 102 scored) vs away team (ID 7) scored 1 (player 205). The stats section lists goals and assists by home ("h") and away ("a") teams.

Player-Specific Endpoints

5. `element-summary/{player_id}/` - Player Detailed Stats

Description: This endpoint returns **detailed information for a single player (element)**, identified by their `player_id` (which you can get from the `elements` list in `bootstrap-static`). It's divided into three main sections for that player ⁵:

- **fixtures:** an array of the player's *upcoming fixtures* for the season ⁵. Each fixture item here includes the opponent team, whether it's home or away, the difficulty rating, and the Gameweek (`event`) number. This lets you see the remaining schedule for the player.
- **history:** a list of the player's *past Gameweek performance* (for the current season) ³⁷. Each history entry corresponds to one fixture the player played in, with fields like: the GW (`round`), opponent team, whether it was home or away, minutes played, goals, assists, clean sheet, cards, and the FPL points scored in that match, etc. Essentially, it's a game-by-game breakdown of the player's season.
- **history_past:** season summaries for *previous years* ³⁸. This includes the player's total points, goals, assists, etc., for each past season they have data (useful for seeing long-term performance).

This endpoint is great for building player detail pages or performing analysis on a player's form and fixture difficulty. For example, a backend service can fetch this to get all past matches of a player for analytics (how they performed against certain teams, etc.), or to list their upcoming matches for planning transfers.

Use case: In a multi-agent system, one agent could fetch `element-summary` for every player to update a database of player histories, or to feed a machine learning model that predicts future performance based on past data. It's also used in apps or websites when you click on a player – you'll often see their recent scores and upcoming games, which come directly from this endpoint.

6. `event/{event_id}/live/` – Live Gameweek Data

Description: The **live endpoint** provides real-time (or recent) data on all players for a specific Gameweek (event) ³⁹. You supply an `event_id` (Gameweek number), and it returns an object that includes a **list of all player IDs and their stats for that gameweek**. Each player entry in the list contains:

- `id`: the player's ID (element) ⁶.
- `stats`: the stats for that player in that gameweek ⁴⁰ – e.g. minutes, goals, assists, clean sheets, saves, bonus, etc. (the exact fields vary by position; for example, goalkeepers have a saves stat, etc.). These stats correspond to the FPL scoring events.
- `explain`: a breakdown of how the player's FPL **points** are calculated from those stats ⁴¹. This is essentially a detailed explanation per player: for instance, it might show "Goals scored: 2 (=> 10 points), Assist: 1 (=> 3 points), Minutes played 72 (=> 2 points), Bonus: 3 (=> 3 points)" – totaling their score. This breakdown is useful for understanding exactly why a player got their points.

This endpoint is used for showing live scores during a gameweek. For example, if matches are ongoing in Gameweek 4, calling `/event/4/live/` will show the up-to-the-minute points for each player (FPL updates this in near real time as goals/assists happen). Even after the GW, this remains the definitive source for final player scores in that round ⁶.

Use cases: Live score tracking and automation. A backend service can poll this endpoint during match days to update the points for players and teams. For instance, a "live rank" website or an AI agent tracking your team's performance would use this to calculate your score as games progress. It's also useful for validating the points after a GW ends (like ensuring your calculations match the official points).

Note: The `explain` section provides granular detail and can be used to, say, identify which players got bonus points or how many points a captain earned (since it would show doubled points for captaincy in the breakdown). This is particularly helpful for debugging or deep analysis.

7. `dream-team/{event_id}/` – Dream Team of the Week

Description: Returns the **Dream Team** for a given Gameweek – essentially the best possible fantasy lineup for that round ⁸. The Dream Team typically consists of the highest scoring players in each position (usually a valid formation with 11 players). The JSON will likely list the players who made the Dream Team and their points, and often includes a `top_player` field which is the single highest scorer of the week ⁸.

While not critical for gameplay, this is a fun endpoint to get the week's top performers. It might include fields like `player_id` and `points` for each spot. Some applications use this for weekly summary content (e.g., "Team of the Week" displays) or to reward users who had many Dream Team players. (*Use case: A content generation agent could use this to write a summary of the GW, highlighting the dream team and top player of the week.*)

8. `stats/top/{statistic}/` – Top Players by Stat

Description: Under the `stats/top/` namespace, FPL provides endpoints to get **top players in various statistical categories** ¹². For example, `stats/top/most_transferred_in/` might give the list of players with the most transfers in (within the current Gameweek), `stats/top/most_selected/` could list players with the highest ownership percentage, and so on. The exact categories available can include: `most_selected`, `most_transferred_in`, `most_transferred_out`, `most_captained`, `most_vice_captained`, etc. These correspond to the “market” and popularity stats often shown on the official site. The data returned would be a list of players and the value of that stat (e.g., number of transfers in). ¹² ⁴²

This is useful for creating “trending players” features or for agents to decide recommendations. For example, a recommendation system might look at `most_transferred_in` players to understand market trends (perhaps to follow the crowd or find differentials). Or a news bot might say “Player X is the most transferred in this week with 200k transfers.”

(Use case: Getting insight into player popularity and market movements. In a multi-agent scenario, one agent might monitor these stats to alert another agent that a certain player’s ownership is spiking, prompting analysis if there’s news like an injury or an upcoming favorable fixture.)

9. `team/set-piece-notes/` – Set Piece Taker Notes

Description: This endpoint provides notes on **which players are the set-piece takers for each team** ⁹. The data is likely structured by team, listing players responsible for penalties, direct free-kicks, indirect free-kicks, and corners. For example, for a given team it might note “Penalties: Harry Kane; Direct Free-Kicks: James Ward-Prowse; Corners: Player A (left), Player B (right)” etc.

This information is useful for fantasy planning because set-piece takers have increased point potential. Use cases include enriching a player info page (show a badge or note if a player is a penalty taker), or for an AI assistant to answer questions like “who is on penalties for Liverpool?”. Backend systems can cache these notes from this endpoint periodically (they don’t change often, but can update if a new player takes over duties or a player leaves).

(Developer note: This is more of a static reference data endpoint. It might be used at season start or when a new player comes in to update knowledge about set piece roles.)

Manager (User/Team) Endpoints

These endpoints deal with data specific to a user’s fantasy team. In FPL, each user is identified by a **manager ID** (also referred to as entry ID or team ID). You can find your own ID by logging into the FPL site – it appears in the URL when viewing your team or history ⁴³. All of the endpoints below require a manager’s ID in the path. Most of them do **not** require authentication if you are querying public data about any team (you can fetch data for other teams by ID), except those specifically noted as requiring login (like `my-team/` and `me/`).

10. `entry/{manager_id}/` – Manager (User) Profile

Description: This returns the **basic profile information** for an FPL team/manager ¹³. The JSON includes fields such as: team name, the player's real name (if not hidden), their current total points, overall rank, favorite club, and other attributes of their team. It also includes a **leagues** section which lists all leagues the team is participating in ¹³ ⁴⁴. In the leagues data, each entry shows the league name, type (s = classic scoring, x = H2H) ⁴⁵, and the manager's current rank and previous rank in that league ⁴⁴. For example, under `leagues -> classic`, you might see the mini-leagues and global leagues the user is in, with their rank in each.

This endpoint is handy for fetching someone's overall performance and basic info. For instance, a multi-league viewer app might call this for each manager in a mini-league to get their total points and show all their leagues. It's also how you'd get someone's overall rank and team name to display on a dashboard ¹³.

Use case: A backend process could iterate through all members of a league (using their IDs) and use `entry/{id}/` to gather their total points and ranks for generating a league report. For multi-agent systems, one agent might handle querying user profiles while another compiles the data for analysis.

11. `entry/{manager_id}/history/` – Manager Season History

Description: Returns the **history of a manager's performance** over time ¹⁴. The response has typically three parts:

- **current:** an array of the manager's scores for each Gameweek so far in the current season ⁴⁶. Each entry includes the GW number, points scored, gameweek rank, overall points total after that week, overall rank after that week, any point hits taken, and the team value etc. This is effectively a week-by-week log of how the team is doing.
- **past:** a list of the manager's results in previous seasons ⁴⁶ (each with season name, final rank, total points, etc.), if the user has played in past years.
- **chips:** which chips (Wildcard, Bench Boost, Free Hit, Triple Captain) have been played and in what gameweek ⁴⁶.

This endpoint is useful for visualizing a manager's progress (e.g., a graph of their rank over the season) or checking their past achievements. A common use is to feed data to a "season history" page or to compare trends among managers.

(Use case: A bot could use this to find out if a rival has used their chips or how many points they've been averaging. Backend analysis might use many users' history data to find patterns like average scores per week, etc.)

12. `entry/{manager_id}/transfers/` – Manager's Transfer History

Description: Provides a list of **all transfers** the manager has made during the current season ¹⁵. Each transfer record typically includes: the Gameweek it was made, the player transferred out, the player transferred in (by their IDs, which can be mapped to names via the bootstrap `elements` data), the cost of the transfer (points hit if it was beyond the free transfer quota), and the date/time of the transfer.

This is extremely useful to track a user's transfer behavior. For example, you might list all the transfers a top player did to analyze their strategy, or have an agent detect if a user brought in a certain player (maybe to

copy that move). It's also used in features like "Transfer history" on some FPL analytics sites which grade your transfers.

(Use case: In a multi-agent scenario, one agent monitoring market trends could also watch top managers' transfer histories by periodically checking this endpoint to see who they brought in, thereby gauging sentiment or strategies.)

Example: A transfer entry might look like: { "event": 5, "element_in": 300, "element_out": 254, "entry": 123456, "time": "2023-09-15T10:30:00Z" } meaning in GW5 the team brought in player ID 300 and sold player ID 254 at that time. (The `entry` field confirms which manager/team made it.)

13. `entry/{manager_id}/event/{event_id}/picks/` – Team Picks for a Gameweek

Description: This endpoint shows the **squad lineup and results for a specific team in a specific Gameweek** ¹⁶. It's the data behind the "Points" page you see for a team in a given week. The response includes four main sections ¹⁶ ⁴⁷:

- **active_chip:** The chip (if any) played that week (e.g., "wildcard", "bench_boost", etc.), or `null` if none ⁴⁸.
- **entry_history:** A summary of the team's performance in that GW ⁴⁸. This includes fields like `points` scored in that event, `total_points` overall after the event, `rank` for the week, `overall_rank`, `bank` (money in the bank), `value` (team value), `event_transfers` made that week and their cost, etc. Essentially, it's the user's Gameweek summary stats.
- **picks:** The list of players in the squad for that Gameweek and their status ⁴⁸. Each pick has: the `element` (player ID), `position` (1-15, where 1-11 are starting XI, 12-15 bench), `multiplier` (points multiplier, 2 for captain, 1 for starting, 0 for benched, 3 if triple captain) ⁴⁹, and flags for `is_captain` or `is_vice_captain`. After the GW is finished, these picks also usually include the points that each player scored that week (often as an added field like `points` or within the `entry_history` breakdown).
- **automatic_subs:** Any automatic substitutions that occurred after the GW (if a starting player didn't play and someone from the bench was subbed in) ⁵⁰. Each entry will state which player came in (`element_in`) for which player went out (`element_out`) and for which event (should match the GW).

This endpoint is crucial for analyzing a team's decision in a given week. For instance, if you want to know **which players someone captained or benched in a particular week**, this has the info. It's also how third-party sites show your team's performance for a week or compare two teams head-to-head in a given GW.

Use cases: A backend service might fetch all teams' picks in a mini-league for the latest GW to compute the league's "dream team" or do an automated head-to-head calculation. Multi-agent systems could use it to simulate matchups (for H2H leagues, for example, comparing two teams' picks). It's also helpful for research, e.g., gathering data on how often certain players are benched or captained across many teams.

Example: Here's an example structure for a team's GW picks (with hypothetical data):

```

{
  "active_chip": null,
  "entry_history": {
    "event": 8,
    "points": 68,
    "total_points": 500,
    "rank": 120000,
    "overall_rank": 150000,
    "bank": 0,
    "value": 1020,
    "event_transfers": 1,
    "event_transfers_cost": 4,
    ...
  },
  "picks": [
    { "element": 254, "position": 1, "multiplier": 2, "is_captain": true,
    "is_vice_captain": false }, // Player 254 (e.g. Salah) was captain
    { "element": 300, "position": 2, "multiplier": 1, "is_captain": false,
    "is_vice_captain": true }, // Player 300 (e.g. Watkins) was vice captain
    { "element": 101, "position": 3, "multiplier": 1, ... }, // starting player
    ...
    { "element": 205, "position": 12, "multiplier": 0, ... }, // bench players
    have 0 multiplier
    ...
  ],
  "automatic_subs": [
    { "entry": 123456, "element_in": 205, "element_out": 102, "event": 8 }
  ]
}

```

In this example, the manager had player 254 as captain (multiplier 2), player 300 as vice, and an automatic sub brought player 205 off the bench for player 102 (who didn't play). The entry_history shows they scored 68 points in GW8, etc.

14. `my-team/{manager_id}/` – Current Squad (Authenticated)

Description: This endpoint returns the **current state of your squad** (the logged-in user's team) ²¹. It requires authentication (you must be logged in and have a valid session token/cookie to access it). The data is similar to the picks endpoint, but specifically for the latest state of your team *before* the next deadline. It typically includes:

- **picks:** your current squad selection (15 players) and who is in starting lineup vs bench, captain/vice, etc. ⁵¹. Essentially your team as it stands for the upcoming or current Gameweek.
- **chips:** which chips you have available or already used (and sometimes which chip is active this week if you've activated one) ⁵². This will list chips like wildcard, freehit, etc., with a status if used.

- **transfers:** information about your transfer status ⁵³ – e.g., how many free transfers you have, any transfer made this week (sometimes last transfer details), and your team value & bank as well.

This is the data behind the “My Team” page when you manage your squad. It’s particularly useful for building third-party transfer planners or squad management tools, because you can pull the live state of your team, including who you have and how much money you have in the bank.

(Use case: A mobile app that lets you view your team would use this to display your squad. In a server context, an AI assistant could use this to give personalized advice, like “you have 1 free transfer and £0.5m in the bank”).

Note: Because this requires auth, automated systems need to handle login (often by using the session cookie from a manual login or a username/password login script). The FPL API uses session-based auth (cookie), not an OAuth token. So typically, you’d log in via a POST to the login form or reuse the web session. The specifics are beyond this scope, but be mindful of not exposing credentials.

15. `me/` – Authenticated User’s Profile

Description: A simple endpoint that returns the profile of the currently authenticated user ²². It’s akin to `entry/{my_id}/` but you don’t need to know your ID; it will return your manager ID (`entry`) and other basic profile info if you are logged in. This is useful right after login – you can call `/api/me/` to get your `manager_id` (entry ID), which you can then use to call other endpoints like `my-team/` or `entry/{id}/history` for yourself ²².

Basically, `me/` is a convenience to confirm your identity and retrieve your user details in one call. It might include username, email, etc., but the main thing is the `entry` field which is your team ID.

(Use case: After an authentication flow, use `/me/` to fetch the user’s FPL team ID so that subsequent calls can be made for their team data. In multi-agent terms, an auth-agent might fetch the cookie and then use this to inform other agents which ID to use for data queries.)

16. `entry/{manager_id}/transfers-latest/` – Latest Transfers (Auth required)

Description: This is a less commonly used, authenticated endpoint which returns the **transfers made in the most recently completed Gameweek** for the given manager ²³. Essentially, after a Gameweek finishes, this will show what transfers the user did leading up to that Gameweek (if any). It’s somewhat similar to the full `transfers/` history, but filtered to just the latest week’s moves. If a gameweek is still ongoing or not all matches are finished, it likely won’t include moves for that GW yet (or might return none until the GW is over).

This could be useful if, for example, you want to quickly check what moves your opponent did this week without pulling the entire transfer list. However, since the public `entry/{id}/transfers/` already lists all transfers, many developers skip this endpoint. It might have been intended for the official site to quickly show “Transfers made this week” in the UI.

(Use case: On a user dashboard, show a summary like “You made 2 transfers for GW10: Out – Salah, In – De Bruyne” by calling this endpoint after the deadline.)

League and Competition Endpoints

FPL has both **Classic leagues** (ranked by total points) and **Head-to-Head (H2H) leagues** (where teams play weekly matches against each other). Additionally, each classic league can have a **League Cup** that usually kicks off midway through the season, and there are overall global competitions. The following endpoints let you retrieve standings and matchups from leagues.

17. `leagues-classic/{league_id}/standings/` - Classic League Standings

Description: Retrieves the standings (table) for a classic scoring league ⁵⁴. The JSON will contain the league details (name, creator, etc.) and an ordered list of entries (teams) with their current rank, team name, manager name, total points, etc. ¹⁷. Each entry typically includes:

- `entry` (the team's ID),
- `player_name` (manager's name),
- `entry_name` (team name),
- `total` (total points),
- `event_total` (points in the current GW),
- and perhaps `last_rank` or movement info.

By default, if a league has many members, the API might not return all entries at once. It often returns the first page (e.g., top 50) ¹⁷. You can use query parameters to paginate:

- `?page_standings={n}` to get subsequent pages of the standings ⁵⁵. For example, page 2 for ranks 51-100, etc.
- There's also `page_new_entries={n}` to page through recently joined teams (the API also provides a list of new entries to the league if the league allows late entries) ¹⁷.
- `phase={phase_id}` can filter the standings by a specific phase of the season (less commonly used; phases are defined in `bootstrap-static` but usually phase 1 is full season).

This endpoint is used whenever you view a mini-league table in apps or websites. It's useful for building custom league views or for aggregating information. For instance, a tool could fetch all pages of a league's standings and then compute additional stats like which teams have the highest team value or who scored highest this week, etc.

(Use case: A backend job could regularly fetch your mini-league's standings to track how ranks change over time, enabling visualizations or sending notifications like "You climbed 2 places in your league this week.")

18. `leagues-h2h/{league_id}/standings/` - Head-to-Head League Standings

Description: Retrieves the table for a **Head-to-Head league** ¹⁸. In H2H leagues, each team earns points based on weekly match results (usually 3 for a win, 1 for a draw). The standings JSON will list each team with fields such as: their number of matches played, wins, draws, losses, and total points in the league, as well as maybe total FPL points scored (some H2H tables show that as a tiebreak). It should also list their rank in the league. The structure is similar to classic standings but with these extra fields for the H2H record ¹⁸.

Like classic leagues, you may need to paginate if the league is large (`page_standings`). The API often defaults to page 1 of standings if not specified.

(Use case: Similar to classic leagues – e.g., assembling a leaderboard for a H2H league. Additionally, one might use this to programmatically determine playoff spots or who's leading in a head-to-head context.)

19. `leagues-h2h-matches/league/{league_id}/?event={gw}` – H2H League Matches

Description: This endpoint provides the **fixtures and results of head-to-head matchups** in a given H2H league for a specific Gameweek ¹⁹. You must specify the `league_id` and usually an `event` (Gameweek number) as a query param. The response will list all the pairings of teams in that league for that week, along with the points each team scored and the result (win/lose/draw). Each match entry might include: team A's entry ID, team B's entry ID, team A's points, team B's points, and who won.

If the league has many teams, the matches might be paginated too (often 10 matches per page, etc.). In that case, you'd use `?page={n}` to get additional matches ⁵⁶. For example, `page=1` might show first 10 matches (if 20 teams, 10 matches), `page=2` might show more if league is larger.

This is how you can programmatically get the *weekly head-to-head results*. It's useful for building custom tools that mimic the official site's H2H schedule and results.

Use case: Suppose you want to create an automated newsletter for your H2H league – you could use this endpoint to grab all the match results each week and then have a script/agent write a summary (“Team A beat Team B 70-65”, etc.). It's also helpful for any analysis of luck in H2H (e.g., who had high scores but still lost because they faced an even higher score). Agents could use this data to find the “unluckiest” team by comparing points vs outcomes.

20. `league/{league_id}/cup-status/` – League Cup Bracket

Description: Many private classic leagues run an internal **knockout cup competition** (usually starting halfway through the season). This endpoint returns data about that cup for a given league ²⁰. It includes the current cup matches or bracket status for that league. For instance, it might show which round is current, which teams are paired, and possibly the results of past rounds or upcoming fixtures. Fields could include the round names, and a list of matches with team IDs and who won.

It's somewhat analogous to H2H matches but for the cup format. If a league doesn't have a cup (e.g., global leagues might not), this might return an empty or null structure.

(Use case: If you wanted to visualize or track the progress of your mini-league's cup tournament, you'd use this. For example, an agent could notify users when they advance to the next round, or a dashboard could display the semifinal matchups.)

21. `stats/best-classic-private-leagues/` – Top 10 Private Leagues

Description: This endpoint gives the **top 10 classic private leagues overall** in FPL ¹⁰. The ranking is determined by the average score of the top 5 teams in each league (this is the same criterion the official site uses to highlight the “Best Leagues”). The JSON likely contains an array of leagues with their names, the average scores, and maybe the league IDs or other metadata.

This is more of a global statistic – not directly useful for managing your team, but interesting for identifying which leagues are performing best. Some people use it to see if their mini-league ranks among the top leagues globally.

(Use case: Content or analytics – e.g., a website might display “League of Power (ID 12345) is currently the top private league with an average of 2100 points”. Not typically needed in multi-agent systems unless you’re specifically tracking league performance as a whole.)

22. `stats/most-valuable-teams/` – Top 10 Most Valuable Teams

Description: Returns a list of the top 5 or top 10 FPL teams by **squad value** (i.e., the total value of the players in the team) ¹¹. FPL managers can increase squad value by players rising in price, so this essentially highlights the “richest” teams in the game. The JSON probably lists the team names or manager along with their squad value.

Again, this is a global stat endpoint. It might not have direct practical use for your team’s decisions, but it could be used for curiosity or content generation (for example, to find out if high team value correlates with success by cross-referencing their ranks).

(Use case: An analytics agent might use this to identify if amassing value early on leads to higher scores later – by perhaps then fetching those teams’ scores. But generally, it’s a novelty metric.)

Conclusion and Practical Considerations

All the above endpoints provide a rich dataset for Fantasy Premier League. In a **developer-friendly** context, you can use these in various backend applications: for example, a web server that regularly pulls data to update a database, or a multi-agent system where different agents focus on different tasks (one agent monitors live scores, another analyzes player stats, another manages user notifications, etc.). Because the API is unofficial, there are a few practical points to note:

- **Rate Limiting & Reliability:** There’s no clearly documented rate limit, but it’s wise to avoid excessive calls (caching static data like `bootstrap-static` and league standings for a few minutes at least). In practice, many tools hit these endpoints regularly without issue, but be mindful especially during high-traffic times (like right after deadlines or during match updates).
- **Data Refresh Timing:** Static data (like player prices in `bootstrap-static`) typically updates once a day or after matches. Live data endpoints update in real-time during games. Event-status is useful to know when final updates are done. Timing your data pulls to these updates ensures you don’t act on incomplete information (e.g., wait for `bonus_added` to be true via `event-status/` before finalizing gameweek points calculations ⁷).
- **Authentication:** For the few endpoints that require login (`my-team/`, `me/`), and performing transfers which is not covered here), you’ll need to handle the FPL login (obtaining the session cookie). The data endpoints we discussed are mostly public. If building a multi-user app (like a mobile app where users log in to see their team), you might have to securely handle their credentials or use the official site’s OAuth if it exists for login (currently, many just simulate the login via POST).

- **Usage in Multi-Agent Systems:** By dividing responsibilities (one agent focuses on league data, one on player stats, one on user-specific data, etc.), you can efficiently utilize these endpoints. For example, an **MCP (Multi-Component Pipeline) server** architecture might have a schedule component that uses `fixtures/` data to queue tasks, a data component that uses `bootstrap-static` to update master data nightly, and a scoring component that pulls `event/{id}/live/` for live updates. Ensuring each component respects the API usage and caches appropriately will lead to a smooth integration.

Finally, while these endpoints are not “officially” supported, they have been stable for years and are widely used by the FPL community ⁵⁷ ⁵⁸. By leveraging them, developers can create rich applications and analyses on top of the Fantasy Premier League game. Always keep an eye on community forums or updated unofficial docs in case the API structure changes with a new season, but overall you now have a **complete guide to the FPL API endpoints** to build your next fantasy tool!

Sources: The information above was compiled from the FPL community’s documentation and examples ² ¹⁷ ¹⁰, including an unofficial FPL API guide by Oliver Looney ⁵⁹ ⁶⁰ and contributions on Reddit ⁶¹ ⁶². Each endpoint’s details were cross-referenced with these sources to ensure accuracy. Enjoy building with the FPL API, and good luck in your leagues!

¹ ⁹ ¹⁵ ²⁰ ²³ ³⁴ ³⁵ ³⁶ ⁴⁶ ⁵⁹ ⁶⁰ **FPL APIs Explained | Oliver Looney**

<https://www.oliverlooney.com/blogs/FPL-APIs-Explained>

² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ¹⁴ ¹⁶ ¹⁷ ²¹ ²² ²⁴ ²⁵ ²⁷ ²⁹ ³⁰ ³¹ ³² ³³ ³⁷ ³⁸ ³⁹ ⁴⁰ ⁴¹ ⁴³ ⁴⁴ ⁴⁵ ⁴⁷ ⁴⁸
⁴⁹ ⁵⁰ ⁵¹ ⁵² ⁵³ ⁵⁴ ⁵⁵ **Fantasy Premier League API Endpoints: A Detailed Guide | by Frenzel Timothy |**

Medium

<https://medium.com/@frenzelts/fantasy-premier-league-api-endpoints-a-detailed-guide-acbd5598eb19>

¹⁰ ¹¹ **Guide: How to access FPL data using Python : r/FantasyPL**

https://www.reddit.com/r/FantasyPL/comments/naduuf/guide_how_to_access_fpl_data_using_python/

¹² ¹³ ¹⁸ ¹⁹ ²⁸ ⁴² ⁶² **FPL API: Is it possible to get live gameweek score, like on the website? : r/FantasyPL**

https://www.reddit.com/r/FantasyPL/comments/1f0fpep/fpl_api_is_it_possible_to_get_live_gameweek_score/

²⁶ ⁵⁶ ⁵⁷ ⁵⁸ ⁶¹ **Cheatsheet of all current FPL Endpoints : r/FantasyPL**

https://www.reddit.com/r/FantasyPL/comments/f8t3bw/cheatsheet_of_all_current_fpl_endpoints/